Distrain: Asynchronous Weighted-Contribution Training of Language Models Across Heterogeneous Devices

Frane Bandov
offbyte
frane@offbyte.com

Abstract

We present Distrain, a fully asynchronous distributed training system for language models on heterogeneous volunteer hardware. Unlike DiLoCo, OpenDiLoCo, and DeMo, which retain synchronous outer optimization steps, Distrain has no synchronization barriers, no quorum requirements, and no agreement step between nodes. Each device trains locally, compresses its weight update via top-k sparsification with error feedback, and pushes the result to a coordinator that merges contributions using staleness-weighted averaging. Every training parameter (batch size, learning rate, compression retention, and push frequency) is auto-calibrated from device capability with no user input. The system is implemented as a single Rust binary that compiles to CUDA, Metal, Vulkan, and CPU from one codebase, eliminating the Python and CUDA toolkit dependencies that make existing systems impractical for volunteer participation.

We empirically characterize the protocol on a 125M parameter transformer across six experiment curves. With baseline hyperparameters matched, distributed training across 3 nodes reaches a loss 1.2 points above single-GPU training (5.98 vs. 4.8 on FineWeb-Edu). We attribute this gap to three addressable sources: top-k compression loss, merge staleness during checkpoint production, and weighted averaging over multiple deltas. The system validates on a 10× throughput gap between Apple Silicon and Intel CPU nodes, and we report negative results on loss-based learning rate scheduling, low-rank delta compression, and data loader state management that inform future protocol design.

1 Introduction

Training large language models requires tightly coupled GPU clusters with low-latency interconnects. Data-parallel SGD with AllReduce [11], ZeRO [20], and FSDP [32] all assume homogeneous hardware, reliable high-bandwidth connections, and synchronous execution. Recent work has begun relaxing these assumptions: DiLoCo [5] reduced communication frequency through inner-outer optimization, OpenDiLoCo [9] scaled this to 10B parameters across continents, and DisTrO [18] reduced communication bandwidth by 857× via decoupled momentum with DCT-based sparsification. All three retain synchronous outer optimization steps, requiring nodes to reach agreement at regular intervals. Slow or intermittent devices get penalized.

We present **Distrain**, a fully asynchronous distributed training system designed for volunteer hardware, in the spirit of SETI@home and Folding@home. Distrain has no synchronization points, no quorum requirements, and no distinction between fast and slow nodes. Each device contributes independently, and the coordinator merges contributions as they arrive. Three design choices distinguish Distrain from prior work: (1) the protocol eliminates outer synchronization entirely, replacing it with staleness-weighted averaging of weight deltas; (2) every training parameter (batch size, learning rate, compression retention, push frequency) is auto-calibrated from device capability,

since volunteer participants will not tune hyperparameters; (3) the system ships as a single Rust binary that compiles to CUDA, Metal, Vulkan, and CPU from one codebase, eliminating the Python and CUDA toolkit setup that makes existing systems impractical for volunteer participation.

We empirically characterize the protocol on a 125M parameter transformer across six experiment curves. The cleanest comparison, matching baseline hyperparameters exactly, shows distributed training across 3 nodes reaching loss 5.98 versus single-GPU 4.8, a 1.2-point gap we attribute to three addressable sources: top-k compression loss, merge staleness during checkpoint production, and weighted averaging over multiple deltas. The system validates on a 10× throughput gap between Apple Silicon GPU and Intel CPU nodes, demonstrating that the protocol works on the heterogeneous hardware mix volunteer compute requires.

Our contributions:

1. An asynchronous weighted-contribution merge protocol that eliminates all synchronization barriers between heterogeneous distributed training nodes.
2. A zero-configuration calibration system that derives every training parameter from device capability, enabling volunteer participation without manual tuning.
3. A single-binary Rust implementation using the Burn framework, deployable on any device with no runtime dependencies.
4. A systematic empirical study identifying the communication-quality tradeoff and characterizing the 1.2-point loss gap into three addressable sources.
5. Negative results on loss-based learning rate scheduling, low-rank delta compression, and data loader state management.

2 Related Work

Synchronous distributed training. Data-parallel SGD with AllReduce is the standard approach for multi-GPU training [6, 11]. FSDP [32] and ZeRO [20] shard optimizer state across devices. These methods require homogeneous hardware, low-latency interconnects, and lock-step synchronization.

Local SGD and federated learning. Local SGD [26, 12] lets workers perform multiple local steps before synchronizing. FedAvg [15] applies this to federated settings. DiLoCo [5] added an outer Nesterov optimizer, achieving strong results with 500× fewer communication rounds. OpenDiLoCo [9] scaled DiLoCo to 10B parameters across multiple continents with 90–95% compute utilization. Their INTELLECT-1 model is the most directly comparable large-scale result: 10B parameters trained across three continents on datacenter GPUs (H100s, A100s) with synchronous outer steps.

Asynchronous SGD. Asynchronous SGD [21, 4] removes synchronization barriers by applying gradient updates as they arrive. Convergence is guaranteed under bounded staleness [7]. Our approach is closest to asynchronous SGD but operates on compressed weight deltas rather than raw gradients.

Gradient compression. Deep Gradient Compression [13] showed that 99.9% of gradient values can be dropped via top-k sparsification without hurting convergence, provided error feedback accumulates the dropped values [27, 10]. QSGD [2] introduced quantized gradient communication. We adopt top-k sparsification with error feedback for delta compression.

Table 1: High-level qualitative comparison of distributed training approaches. Categories compress nuance; see text for details.

	Sync free?	Hetero. HW?	Consumer HW?	Single binary?	Auto-config?	Scale tested
AllReduce/FSDP	×	×	×	×	×	1000s GPUs
DiLoCo	×	×	×	×	×	8–64 GPUs
OpenDiLoCo	×	×	×	×	×	10B, 3 continents
DisTrO/DeMo	×	partial	partial	×	×	32 H100s, 1B
Hivemind	×	✓	✓	×	×	100s GPUs
Distrain	✓	✓	✓	✓	✓	**3 nodes, 125M**

Decentralized training platforms. Hivemind [22] provides a library for decentralized training, used mainly for inference via Petals [3]. SWARM parallelism [23] showed that large model training can be surprisingly communication-efficient. DisTrO [18] reduces communication bandwidth by 857× using decoupled momentum with DCT-based sparsification, but retains synchronous all-reduce on the compressed components. Their Psyche platform coordinates training via Solana blockchain [16]. Our approach eliminates the all-reduce entirely: nodes push independently with no agreement step.

Table 1 summarizes the key differences.

3 Method

3.1 Weighted Contribution Protocol

Let θ_v denote the model parameters at checkpoint version v. A node i that trains against checkpoint θ_v for h_i steps produces a delta:

$$\delta_i = \theta_v^{(i)} - \theta_v \tag{1}$$

where $\theta_v^{(i)}$ are the parameters after local training. Each delta encodes gradient information that has been processed by the local optimizer. Rather than transmitting raw gradients (which would require communication after every step), each node runs h_i steps of AdamW locally and sends the resulting weight change. The delta captures the accumulated effect of multiple gradient updates, including AdamW's momentum, adaptive per-parameter learning rates, and weight decay. This is the same design principle as DiLoCo's "pseudo-gradients" [5]. The alternative, sending compressed gradients every step as in DeMo [18], preserves more signal per communication but requires 50× more frequent transmission, making it impractical for volunteer devices on residential internet.

Each delta is assigned a weight:

$$w_i = T_i \cdot \gamma^{s_i} \tag{2}$$

where T_i is the number of tokens processed by node i in this round, $\gamma = 0.9$ is the staleness decay factor, and $s_i = v_{\text{current}} - v_i$ is the staleness. Deltas with $s_i > 10$ are discarded. Using tokens rather than steps ensures that nodes with different batch sizes are weighted proportionally to their data contribution.

The coordinator produces new checkpoints by weighted averaging:

$$\bar{\delta} = \frac{\sum_i w_i \cdot \text{decompress}(\delta_i)}{\sum_i w_i} \tag{3}$$

$$\theta_{v+1} = \theta_v + \bar{\delta} \tag{4}$$

Simple weighted averaging without momentum or learning rate scaling produced the best results, reducing the outer optimizer to direct delta application ($\eta_{\text{outer}} = 1.0$, $\mu = 0$). Early experiments (Curves B–C) used adaptive outer learning rates in the range $[0.07, 0.97]$, which discarded up to 93% of the merged delta's magnitude. In our experiments, treating the merged delta as the full outer update worked best; scaling it below 1.0 consistently hurt convergence.

Merge properties. The weighted averaging is commutative and associative within each accumulation window. Deduplication by (node_id, sequence number) prevents duplicate application. Our merge protocol draws on ideas from conflict-free replicated data types [19], specifically commutativity and deduplication, though we do not claim formal CRDT properties as the accumulator resets at checkpoint boundaries.

Checkpoint production. When the accumulator has received contributions from at least half of active nodes, the coordinator produces a new checkpoint via a two-phase compare-and-swap commit on object storage. A node is considered active if it has pushed a delta within the last 3 checkpoint cycles. The minimum trigger is 2 contributions, preventing single-node checkpoint flooding (a failure mode we encountered when a fast node produced checkpoints alone, wasting other nodes' concurrent work). Both the active node count and the minimum are computed automatically.

3.2 Delta Compression

Raw model deltas in BF16 are prohibitively large for consumer internet connections (14 GB for a 7B model). Our compression pipeline:

1. **Adaptive top-k sparsification.** We retain the top k% of delta values by magnitude. Early in training, when gradients are concentrated, we use $k = 0.5$%. As gradients become more diffuse, k increases to 7%.

2. **Error feedback.** Values dropped by sparsification accumulate in a local error buffer e_i. On the next push, the effective delta becomes $\delta_i + e_i$ before sparsification is applied. As a concrete example: at 20% retention, a node pushes the top 20% of its delta in round 1 and stores the remaining 80% in the error buffer. In round 2, the buffer is added to the new delta before sparsification, so values that were small in round 1 but grew in round 2 enter the top-k set. Over 3–5 rounds, most gradient signal cycles through. However, the error buffer resets when a new checkpoint is produced. Any signal still in the buffer at that point is lost. We estimate that 60–80% of total gradient signal reaches the coordinator over each checkpoint cycle, based on typical retention rates (20%) and checkpoint frequency (3–4 rounds per cycle). Precise measurement of cumulative signal retention is left to future work. The error feedback guarantee [10] holds only within a single accumulation window.

3. **Sorted-index encoding + zstd.** Sparse indices are sorted and delta-encoded, then the entire payload is compressed with zstd.

For the 125M model at 90% retention, this produces ∼300 MB deltas. At 1–2% retention, deltas shrink to ∼5 MB. The system automatically selects retention based on measured upload bandwidth: datacenter nodes send near-raw deltas, residential nodes use higher compression.

3.3 Zero-Configuration Device Calibration

For volunteer computing, manual configuration is a non-starter: participants will not tune batch sizes, learning rates, or compression parameters. Every training parameter must be determined automatically from the device's hardware.

- **Compute backend.** The node probes for GPU capability (CUDA, then Metal/Vulkan via wgpu) and falls back to CPU if no GPU is available or if the GPU has insufficient memory. No user input required.
- **Batch size** is computed from GPU VRAM and model architecture (hidden dimensions, layers, FFN width, vocabulary size, sequence length). On out-of-memory, the batch size is halved and training retries automatically. An A40 with 48 GB gets batch = 26 for the 125M model; an M2 Pro with 32 GB unified memory gets batch = 4.
- **Inner steps** (h_i) target approximately 60-second push intervals:

$$h_i = \text{clamp}\left(\left\lfloor \frac{T_{\text{target}}}{t_{\text{step}}} \right\rfloor,\ h_{\min},\ h_{\max}\right) \tag{5}$$

  where $T_{\text{target}} = 60\,\text{s}$, t_{step} is the measured seconds per step, $h_{\min} = 50$, and $h_{\max} = 500$. After the first round, h_i is refined based on actual upload throughput.
- **Compression retention** is selected from measured upload bandwidth. On a datacenter connection (50+ MB/s), the node sends near-raw deltas (90% retention). On residential broadband (1 MB/s), it uses 2% retention. The target is to keep upload time under 10 seconds.

To our knowledge, no existing distributed training system auto-calibrates batch size, learning rate, compression, and push frequency from hardware alone. Existing systems typically require manual specification of these parameters. For volunteer compute, zero-configuration participation is not a convenience but a requirement.

3.4 Shard Assignment

Training data is split into fixed-size shards (10M tokens each). Each node computes its shard assignment deterministically:

$$\text{seed} = \text{hash}(\text{node_id},\ v) \tag{6}$$

Different nodes receive different subsets, maximizing data coverage across the network. No coordinator involvement is needed.

4 System Implementation

4.1 Architecture

The system has two components: a stateless coordinator and an arbitrary number of training nodes. Both are implemented in Rust using the Burn deep learning framework [25] for tensor operations and automatic differentiation. Burn is backend-agnostic: the same model code compiles to CUDA (NVIDIA), Metal (macOS/iOS via wgpu), Vulkan (Linux/Android via wgpu), and CPU (ndarray). All persistent state resides in S3-compatible object storage (Cloudflare R2 in production, MinIO

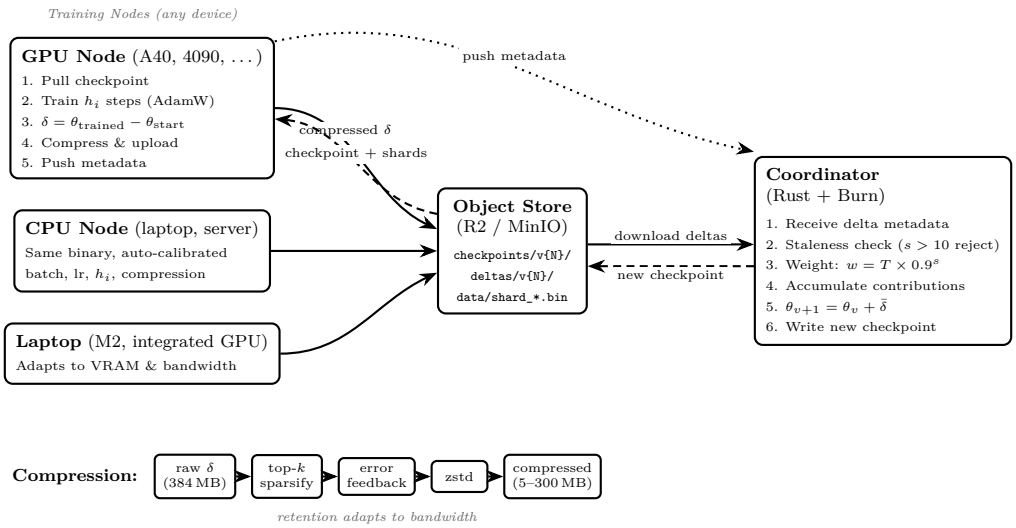

Figure 1: System architecture. Nodes train independently, compress deltas via top-k sparsification with error feedback, and push to object storage. The coordinator merges contributions via staleness-weighted averaging. No synchronization barriers exist between nodes.

for local development). Checkpoints use the safetensors format. Training data is stored as binary uint16 token shards. Docker images for both coordinator and node are built via GitHub Actions.

The **coordinator** is an HTTP server built on Axum and Tokio with five endpoints: node registration, delta push, checkpoint query, status, and health. Aggregation (decompression, weighted averaging, checkpoint production) uses Burn with the ndarray CPU backend. The coordinator is stateless, has no Python dependency, and can be restarted without data loss.

The **training node** is a single static binary that handles calibration, data loading, training, compression, and communication. Burn selects the GPU backend at runtime based on what the device supports. The node runs a continuous loop: pull checkpoint, train, compress delta, push, repeat. Compression runs asynchronously off the GPU thread so the GPU never idles waiting for upload.

Every other system in our related work (DiLoCo, OpenDiLoCo, DisTrO, Hivemind) requires Python and PyTorch. For a volunteer computing system, this matters: PyTorch means a 2+ GB install, Python version conflicts, CUDA toolkit matching, and environment setup. Distrain is one ∼20 MB binary that runs immediately. Figure 1 shows the full system.

4.2 Model

We implement a standard decoder-only transformer [30] following current best practices: Grouped-Query Attention [1] with RoPE [28] ($\theta = 500{,}000$), SwiGLU feed-forward network [24], RMSNorm [31] (pre-norm), and tied embeddings with learned output scaling, following LLaMA [29]. The optimizer is AdamW [14]. The tokenizer is Mistral v0.3 (vocabulary size 32,768).

We evaluate two model sizes:

- **Tiny** (125M parameters): 768 hidden, 12 layers, 12 attention heads, 4 KV heads.
- **Small** (1.13B parameters): 2048 hidden, 24 layers, 16 attention heads, 4 KV heads.

4.3 Training Data

We use FineWeb-Edu [17], tokenized with the Mistral v0.3 tokenizer and split into shards of 10M tokens each (∼20 MB per shard in uint16 binary format). The full dataset comprises 1,102 shards totaling 11.01 billion tokens.

5 Experiments

5.1 Setup

Datacenter experiments (Curves B–F). For controlled comparison, we use 3× NVIDIA A40 (48 GB VRAM) on RunPod, with the coordinator on a separate pod. The single-GPU baseline (Curve A) uses 1× RTX 4000 Ada with the same model, data, and hyperparameters (batch = 4, lr = 3e−4, cosine schedule).

Consumer hardware validation. Before running controlled datacenter experiments, we validated that the protocol works at all on heterogeneous consumer hardware. Three laptops (two Apple Silicon GPUs and one Intel CPU) trained together for 2,928 checkpoints over 46.7M tokens, reaching loss 6.7 despite a 10× throughput gap between the fastest and slowest device. The run confirmed that fully asynchronous training converges on diverse consumer devices. The limited token budget precludes meaningful comparison with the datacenter experiments, so we report it separately.

Device	Processor	Memory	Backend	Throughput
MacBook Pro 14"	Apple M2 Pro	32 GB unified	GPU (Metal)	∼1,024 tok/s
MacBook Air 13"	Apple M2	24 GB unified	GPU (Metal)	∼800 tok/s
MacBook Pro 13"	Intel i7-1068NG7	32 GB	CPU	∼100 tok/s

5.2 Systematic Optimization (Curves B–F)

We ran six experiment curves, each motivated by the previous result. All distributed runs used 3 nodes. The baseline used 1 GPU with the same model, data, and hyperparameters. Results are in Table 2 and Figure 2.

Curve B (plateau 6.4). Our first distributed run on datacenter GPUs. We changed the outer learning rate to 1.0 (up from the adaptive range used on consumer hardware) and tuned compression retention to 40–65%. The gap to baseline was 1.6 points. Many variables changed at once, so we could not attribute the gap to any single factor, but the outer LR change was the biggest win: treating the merged delta as the full update worked best, and scaling it below 1.0 had been throwing away signal.

Curve C (plateau 6.0). We added patience-based checkpoint triggers: instead of producing a new checkpoint as soon as any single node pushes, the coordinator waits until at least half of active nodes have contributed. More diverse deltas per checkpoint means a better weighted average. The gap dropped from 1.6 to 1.2. C's initial spike to 121.1 at 0.5M tokens reflects a poor first checkpoint produced from limited early contributions before the patience-based triggering stabilized. This confirmed that contribution diversity matters more than checkpoint frequency.

Table 2: Experiment curves on the 125M transformer. Gap is plateau loss minus baseline (4.8). Perplexity = exp(loss). Wall-clock times: Curve A ran ∼6 hours on 1× RTX 4000 Ada; each datacenter curve (B–F) ran ∼3 hours on 3× A40; the consumer hardware run took ∼10 days on 3 laptops. Total compute cost for all datacenter experiments: ∼$25 at RunPod rates ($0.20/hr for RTX 4000 Ada, $0.44/hr for A40).

Curve	Hardware	Loss	PPL	Gap	Tokens	Key Change
A (BL)	1× RTX 4000 Ada	4.8	121.5	—	∼35M	Single GPU, batch=4, lr=3e-4
B	3× mixed RTX	6.4	601.8	+1.6	∼30M	Outer LR→1.0, top-k tuning
C	3× RTX A4000	6.0	403.4	+1.2	∼30M	Patience triggers
D	3× A40	5.8	330.3	+1.0	∼30M	Continuous training
E	3× A40	5.98	395.4	+1.2	∼30M	Persistent data loader, constant lr
F	3× A40	6.4	601.8	+1.6	∼17M	Loss-based lr (negative result)

Table 3: Loss at matched token counts. Values marked † indicate plateau. Validation loss on held-out data matches training loss within 0.4%.

Tokens	A (BL)	B	C	D	E	F
0.5M	47.5	48.7	121.1	30.6	52.5	32.7
1M	30.5	43.0	44.7	30.6	31.0	25.3
2M	23.5	32.5	33.8	13.8	18.4	15.2
3M	16.3	25.5	24.9	13.8	12.3	10.9
5M	9.4	16.9	14.4	7.9	8.6	7.8
7M	7.2	12.6	10.3	7.9	7.3	6.8
10M	5.3	9.0	8.1	6.1	6.8	6.9
15M	5.9	7.3	7.0	6.1	6.4	6.4†
20M	5.3	6.8	6.6	5.8	6.2	—
30M	5.1	6.5	6.3	—	5.98†	—

Curve D (plateau 5.8). We kept the GPU busy between rounds: no idle time for checkpoint downloading or shard reloading. Compression moved off the GPU thread, running asynchronously. The gap dropped to 1.0, our best number. But D changed multiple things at once (continuous training, compression settings, GPU scheduling), so the number mixes protocol cost with engineering optimizations.

Curve E (plateau 5.98). We ran E specifically because D's number was not cleanly attributable. The question: what does the distributed protocol cost when everything else matches the baseline? Same batch = 4, same lr = 3e−4, same cosine schedule. The only variable is distributed vs. single GPU. The answer: 1.2 points. D achieved 5.8 (the lowest loss), but E provides the cleaner measurement. We report E's 1.2-point gap as our headline result because it isolates the protocol cost from engineering optimizations. We also observe that 3-contribution checkpoints consistently produce lower loss than 2-contribution ones, confirming that diverse gradients improve merge quality.

Curve F (plateau 6.4). We ran F to test the hypothesis that the inner learning rate should decay as loss drops: $\text{lr} = \text{lr}_{\max} \times \text{clamp}(\text{loss}_{\text{ema}}/\ln V,\ 0.1,\ 1.0)$. The idea is sound: learn fast when loss is high, refine carefully as the model converges. F did converge faster than E early on (loss 7.8 vs. 8.6 at 5M tokens). But the adaptive LR decayed before the model had finished learning, and F plateaued 0.4 points higher than E (6.4 vs. 5.98). Constant lr is more robust for this protocol.

An additional excluded experiment attempted auto-tuned batch size (batch = 25) with linear LR scaling. A data loader bug caused silent memorization (§5.4). The failure led us to discover that

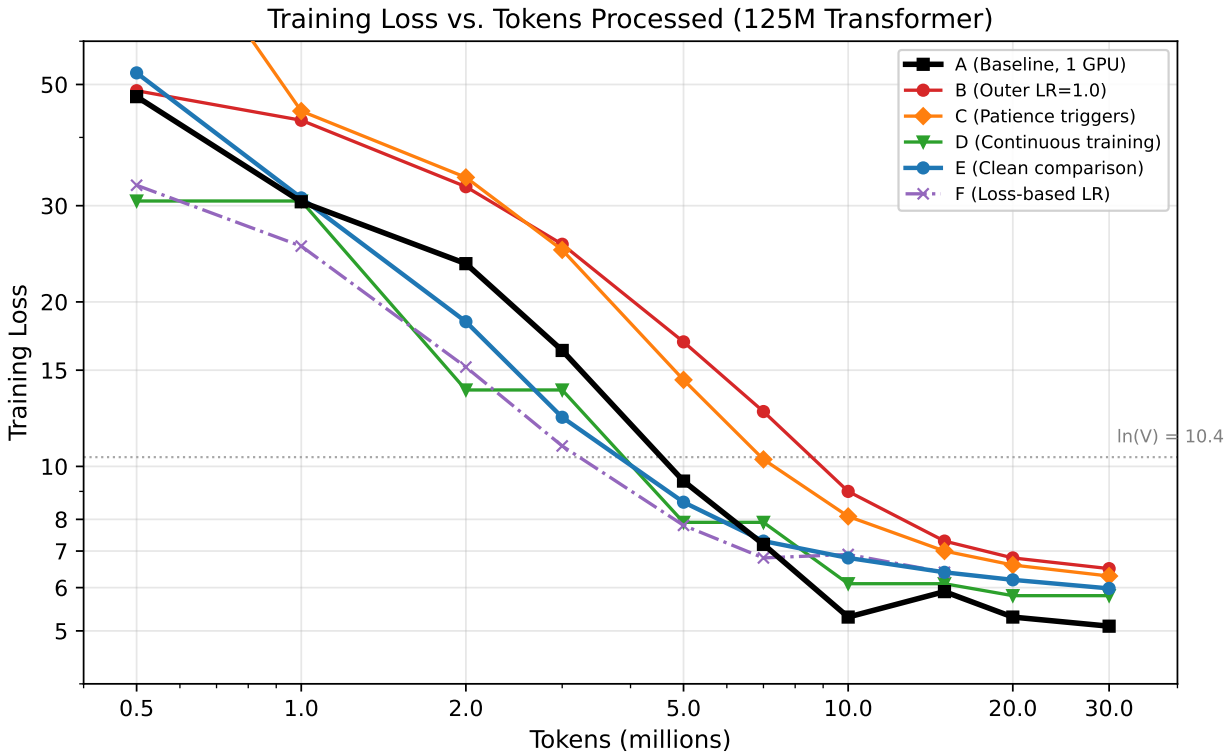

Figure 2: Training loss vs. tokens processed for all experiment curves. The baseline (A, thick black) trains on a single GPU. All distributed curves use 3 nodes. The dashed horizontal line marks $\ln(32{,}768) \approx 10.4$, the loss of a uniform random predictor. Curve E (blue) is our cleanest comparison, matching baseline hyperparameters exactly.

batch = 4 gives better per-token efficiency (§5.3) and that data loader state must persist across checkpoint changes.

Validation. We evaluated final checkpoints on held-out data (shards not used during training). Validation loss matched training loss within 0.4% (6.38 validation vs. 6.4 training on Curve F), confirming that the persistent data loader eliminates data repetition. At <2% data utilization (28M tokens consumed from 2.2B available per node), training loss is a reliable proxy for generalization.

5.3 Batch Size and Token Efficiency

An excluded experiment used auto-detected batch = 25 on A40 GPUs (48 GB VRAM) with linear LR scaling ($\text{lr} = 3\text{e}{-4} \times 25/4 = 1.875\text{e}{-3}$). It converged faster in wall-clock time but slower per token than batch = 4:

Batch	LR	Tokens/step	Gradient updates per 1M tokens
4	3e-4	2,048	488
25	1.875e-3	12,800	78

Smaller batch = more gradient updates per token = better per-token convergence. The linear scaling rule compensates update magnitude but not update frequency. All reported curves use batch = 4.

5.4 Negative Results

Loss-based inner LR (Curve F). The adaptive LR $\mathrm{lr} = \mathrm{lr}_{\max} \times \mathrm{clamp}(\mathrm{loss}_{\mathrm{ema}}/\ln V,\ 0.1,\ 1.0)$ gives full learning rate while loss exceeds $\ln V \approx 10.4$, then decays smoothly. F converged faster than E early (7.8 vs. 8.6 at 5M tokens) but plateaued at 6.4 versus E's 5.98. The decay kicked in too early, leaving 0.4 points on the table. Constant lr = 3e−4 is more robust.

Data loader state persistence. In earlier experiments (before Curves E and F), the data loader reset to position 0 on every checkpoint change, causing the model to repeatedly train on the same initial tokens. The result was apparent convergence: training loss dropped to near-zero while the model memorized rather than generalized. This failure mode is silent. All training metrics look healthy. We detected it only by comparing training loss to held-out validation loss. The fix is simple: persist data loader state across checkpoint changes, never resetting position. All curves in the comparison table use the persistent data loader. This bug generalizes beyond our system: any distributed training protocol that reloads data state on checkpoint changes is vulnerable to the same silent memorization.

Low-rank delta compression. Rank-32 decomposition of training-from-scratch deltas gave 88–98% reconstruction error. Pre-training deltas are full-rank: diverse data through AdamW [14] produces gradient updates spanning many dimensions simultaneously. Low-rank compression works for fine-tuning [8] where changes are structured, not for pre-training. Top-k sparsification with error feedback was the only method that worked in our experiments.

5.5 Communication–Quality Tradeoff

The best training quality requires high-retention deltas, but these are large:

Connection	Upload	Delta Size	Retention	Upload Time
Datacenter	50+ MB/s	∼300 MB (top-k 90%)	90%	∼6s
Fast residential	10 MB/s	∼50 MB (top-k 20%)	20%	∼5s
Slow residential	1 MB/s	∼5 MB (top-k 2%)	2%	∼5s

Curves B–D showed that lower compression retention costs 1.0–1.6 loss points versus the baseline. The bandwidth-adaptive system selects the best compression for each node's connection automatically.

5.6 Preliminary 1B Scaling

We ran a preliminary 1B parameter model (2048 hidden, 24 layers, 16 attention heads, 4 KV heads) on 3 heterogeneous consumer devices (M2 Pro GPU, M2 GPU, Intel i7 CPU) over 5 days. Loss dropped from 1,669 (random init) to 48.8 over 63 checkpoints, consuming approximately 3.5M tokens.

This run predates several bug fixes (persistent data loader, error buffer persistence) and lacks a single-GPU baseline, so we do not include it in the systematic analysis. It does indicate that the protocol scales to 1B parameters: the coordinator merged 4.5 GB checkpoints, compression reduced deltas to ∼25 MB (175× compression), and all three devices contributed despite a 10× throughput gap. This run, while not a controlled comparison, demonstrates that the protocol's compression and merge mechanisms scale to checkpoint sizes 36× larger than 125M without additional engineering.

Rigorous evaluation at 1B scale, with a matched single-GPU baseline and controlled hyperparameters, is an important direction for future work.

5.7 Generalization Across Model Sizes

The protocol is in principle independent of model size, operating on weight deltas regardless of architecture. Our preliminary 1B experiments show qualitatively similar convergence behavior. However, larger models produce proportionally larger deltas, and the communication-quality tradeoff at 7B and beyond remains empirically unvalidated. At 7B, raw deltas are 14 GB, requiring aggressive compression (top-k 1–2%) on residential connections. Whether the communication-quality tradeoff remains favorable at 7B depends on node count and available bandwidth. With enough nodes contributing high-retention deltas on fast connections, the protocol should scale; with few nodes on slow connections, the compression loss may dominate.

5.8 Qualitative Output

At loss ∼7 (mid-training), the model produces English words with correct spacing and no control token leakage. Content is incoherent, consistent with a 125M model at this training stage. A 125M model trained on 30M tokens is too small to produce coherent text by modern standards, but the output confirms the distributed protocol trained a functional language model, not noise.

6 Discussion

Convergence. Our protocol shares structural similarities with stale synchronous parallel methods [7]: bounded staleness, weighted contribution averaging, and error feedback for compression. However, existing convergence theorems do not directly apply. Nodes push multi-step weight deltas (the accumulated effect of h_i AdamW steps), not single-step gradient estimates. The error feedback buffer resets at checkpoint boundaries, breaking the guarantee that no gradient information is permanently lost [10]. The coordinator merges contributions in discrete checkpoint cycles with weighted averaging, rather than applying updates continuously. We provide empirical evidence of convergence: validation loss matches training loss within 0.4%, loss decreases monotonically across all experiment curves, and the protocol converges across heterogeneous devices with a 10× throughput gap. Formal convergence analysis is an important direction for future work.

The delta size problem. The best training quality requires high-retention deltas, but these are large.

Model	Raw Delta	Top-k 90% + zstd	Top-k 1% + zstd	Residential (10 Mbps)
125M	384 MB	∼300 MB	∼5 MB	6s (1%) – 2.3 min (raw)
1B	4.5 GB	∼3.5 GB	∼50 MB	40s (1%) – 60 min (raw)
7B	14 GB	∼11 GB	∼200 MB	2.7 min (1%) – 3.1 hr (raw)

The protocol works best when deltas are large, but volunteer computing requires small deltas. The bandwidth-adaptive system mitigates this by selecting compression per node, but the tension is real.

Is the 1.2-point gap fundamental? Likely not. Three sources contribute to it: compression loss from top-k sparsification, merge staleness during checkpoint production, and averaging multiple deltas instead of direct gradient application. Each is addressable (§6.1). Higher compression retention closes the gap (Curve D achieved 1.0 with 90% retention vs. Curve E's lower retention). More contributors per checkpoint improve merge quality (3-contribution checkpoints consistently outperform 2-contribution ones in our data). GPU-accelerated aggregation would reduce merge staleness from 30 seconds to under 1 second. We expect the gap to narrow with more nodes, higher bandwidth, and longer training, but confirming this requires experiments at larger scale.

Why low-rank compression fails for pre-training. A natural hypothesis: training deltas, like fine-tuning updates, occupy a low-dimensional subspace and could be compressed via low-rank decomposition [8]. We implemented randomized SVD and tested it on pre-training deltas. Rank-32 decomposition of 768-dimensional weight matrices yielded 88–98% reconstruction error.

Pre-training deltas are full-rank. Each round processes diverse data through AdamW, producing updates that span many dimensions. Fine-tuning is different: a pre-trained model makes small, structured adjustments that naturally lie in a low-rank subspace. Top-k sparsification with error feedback is, to our knowledge, the only effective compression for pre-training deltas; low-rank methods that work well for fine-tuning fail in this regime.

Error feedback at checkpoint boundaries. Error feedback persists within a checkpoint version but resets at boundaries. Gradient signal in the buffer is lost when a new checkpoint is produced. The error feedback guarantee [10] holds only within a single accumulation window. In practice, with $\sim$50 inner steps per round and 2–3 rounds per checkpoint, the buffer has limited time to recover dropped values.

Scale. Our experiments use 3 nodes. The protocol is designed for 50 to 1,000+ nodes, but staleness dynamics, merge quality, and coordinator throughput at that scale remain untested. With 3 nodes, every contribution matters and the merge is dominated by a few deltas. At 100+ nodes, the merge averages over many contributions, staleness distributions widen, and the coordinator must handle higher throughput. Validating the protocol at larger scale, with diverse real-world network conditions and device populations, is the most important next step.

Memory. Training 7B requires 20–40 GB VRAM minimum. Most consumer devices cannot participate at frontier model sizes without activation checkpointing or expert sharding (MoE).

Single coordinator. The current architecture has one coordinator. A peer-to-peer topology using gossip-based state replication is a natural extension.

Cost. The entire experimental campaign (baseline + five distributed curves) cost approximately \$25 in cloud GPU rental at RunPod rates. Individual curves cost \$1–4 each. For comparison, training a 7B model on 1T tokens on a dedicated A100 cluster costs \$100,000–200,000 in cloud compute. The volunteer model distributes this cost across participants, each contributing idle compute worth a few dollars in electricity. Whether this tradeoff is practical depends on the communication-quality gap at larger scales.

Other applications. While we focus on volunteer pretraining, the protocol applies to any setting with heterogeneous or intermittent compute, such as federated learning across organizational boundaries or opportunistic use of idle cluster resources.

6.1 Future Directions

Staleness recovery. The current protocol discards deltas older than 10 checkpoint versions and exponentially down-weights stale contributions. A stale delta still contains gradient signal from data the current model may not have seen. We plan to investigate delta rebasing: the coordinator subtracts the accumulated model drift from a stale delta before merging, preserving the novel component while discarding the redundant signal. For extremely stale contributions, an alternative is proxy replay: the coordinator forwards the slow node's data selection (shard IDs and offsets) to a fast node, which retrains on that data against the current checkpoint. This recovers the slow node's data coverage without its parameter staleness. A staleness cascade (direct merge for fresh deltas, rebasing for moderate staleness, proxy replay for extreme staleness) could reduce the effective cost of heterogeneity.

Structured compression. Our current pipeline selects individual values by magnitude (unstructured top-k), producing randomly scattered indices that are expensive to encode. We plan to investigate structured block sparsity: selecting entire rows or blocks of weight matrices by their aggregate norm, then applying per-block INT8 quantization with independent scale factors. This could reduce index overhead from $O(k)$ individual uint32 indices to $O(k/d)$ block indices, where d is the block width, while preserving gradient coherence within each block. Combined with importance-weighted selection (prioritizing parameters that have moved least recently over those with the largest instantaneous magnitude), this could improve both compression ratio and signal quality.

Scale validation. We plan a public volunteer campaign to validate the protocol beyond 3 nodes, measuring: how the convergence gap changes with node count, the staleness distribution under real-world heterogeneity and network conditions, and the coordinator's throughput under concurrent delta pushes. We also plan a clean 1B parameter experiment with a matched single-GPU baseline, addressing the main experimental limitation of this work.

7 Conclusion

We built and empirically characterized a fully asynchronous training system for language models on heterogeneous volunteer hardware. It works, it is simple, and it exposes a real communication-quality tradeoff.

Through six experiment curves on a 125M parameter model, we identified a 1.2-point loss gap between distributed (3 nodes) and single-GPU training when hyperparameters are matched (Curve E). The gap narrows to 1.0 points with engineering optimizations (Curve D). Three sources contribute: compression loss from top-k sparsification, merge staleness during checkpoint production, and averaging multiple deltas instead of direct gradient application. Each is addressable, and the gap is likely not fundamental.

We report negative results that are equally informative: loss-based inner LR (Curve F) plateaus 0.4 points higher than constant LR (Curve E); low-rank delta compression gets only 2–12% signal

retention on pre-training deltas; and data loader state resets cause silent memorization. These failures shaped the final protocol as much as the successes.

The system is implemented as a single Rust binary using the Burn framework [25], compiling to CUDA, Metal, Vulkan, and CPU from one codebase. A volunteer downloads one file and starts contributing. No Python, no package manager, no environment setup. Validating the protocol beyond 3 nodes, on larger models, and with real-world network conditions is the most important next step. Source code is available at https://github.com/frane/distrain.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, December 2023. URL http://arxiv.org/abs/2305.13245. arXiv:2305.13245 [cs].

[2] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding, December 2017. URL http://arxiv.org/abs/1610.02132. arXiv:1610.02132 [cs].

[3] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative Inference and Fine-tuning of Large Models, March 2023. URL http://arxiv.org/abs/2209.01188. arXiv:2209.01188 [cs].

[4] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc V Le, and Andrew Y Ng. Large Scale Distributed Deep Networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.

[5] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc'Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. DiLoCo: Distributed Low-Communication Training of Language Models, September 2024. URL http://arxiv.org/abs/2311.08105. arXiv:2311.08105 [cs].

[6] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour, April 2018. URL http://arxiv.org/abs/1706.02677. arXiv:1706.02677 [cs].

[7] Qirong Ho, James Cipar, Henggang Cui, Seunghak Lee, Jin Kyu Kim, Phillip B Gibbons, Garth A Gibson, Greg Ganger, and Eric P Xing. More Effective Distributed ML via a Stale Synchronous Parallel Parameter Server. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, October 2021. URL http://arxiv.org/abs/2106.09685. arXiv:2106.09685 [cs].

[9] Sami Jaghouar, Jack Min Ong, and Johannes Hagemann. OpenDiLoCo: An Open-Source Framework for Globally Distributed Low-Communication Training, July 2024. URL http://arxiv.org/abs/2407.07852. arXiv:2407.07852 [cs].

[10] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian U. Stich, and Martin Jaggi. Error Feedback Fixes SignSGD and other Gradient Compression Schemes, May 2019. URL http://arxiv.org/abs/1901.09847. arXiv:1901.09847 [cs].

[11] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. PyTorch distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13 (12), 2020. doi: 10.14778/3415478.3415530.

[12] Tao Lin, Sebastian U. Stich, Kumar Kshitij Patel, and Martin Jaggi. Don't Use Large Mini-Batches, Use Local SGD, February 2020. URL http://arxiv.org/abs/1808.07217. arXiv:1808.07217 [cs].

[13] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J. Dally. Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training, June 2020. URL http://arxiv.org/abs/1712.01887. arXiv:1712.01887 [cs].

[14] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, January 2019. URL http://arxiv.org/abs/1711.05101. arXiv:1711.05101 [cs].

[15] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data, January 2023. URL http://arxiv.org/abs/1602.05629. arXiv:1602.05629 [cs].

[16] Nous Research. DisTrO: A preliminary report. https://github.com/NousResearch/DisTrO/blob/main/A_Preliminary_Report_on_DisTrO.pdf, 2024.

[17] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale, October 2024. URL http://arxiv.org/abs/2406.17557. arXiv:2406.17557 [cs].

[18] Bowen Peng, Lizhang Chen, Baiyu Su, Jeffrey Quesnelle, Diederik P. Kingma, and Qiang Liu. DeMo: Decoupled Momentum Optimization, February 2026. URL http://arxiv.org/abs/2411.19870. arXiv:2411.19870 [cs].

[19] Nuno Preguiça, Carlos Baquero, and Marc Shapiro. Conflict-free Replicated Data Types (CRDTs), May 2018. URL http://arxiv.org/abs/1805.06358. arXiv:1805.06358 [cs].

[20] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, May 2020. URL http://arxiv.org/abs/1910.02054. arXiv:1910.02054 [cs].

[21] Benjamin Recht, Christopher Re, Stephen Wright, and Feng Niu. Hogwild: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. In *Advances in Neural Information Processing Systems*, volume 24, 2011.

[22] Max Ryabinin and Anton Gusev. Towards Crowdsourced Training of Large Neural Networks using Decentralized Mixture-of-Experts, October 2020. URL http://arxiv.org/abs/2002.04013. arXiv:2002.04013 [cs].

[23] Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. SWARM Parallelism: Training Large Models Can Be Surprisingly Communication-Efficient, June 2023. URL http://arxiv.org/abs/2301.11913. arXiv:2301.11913 [cs].

[24] Noam Shazeer. GLU Variants Improve Transformer, February 2020. URL http://arxiv.org/abs/2002.05202. arXiv:2002.05202 [cs].

[25] Nathaniel Simard, Louis Fortier-Dubois, Dilshod Tadjibaev, Guillaume Lagrange, and Burn Framework Contributors. Burn: A flexible and comprehensive deep learning framework in Rust. https://github.com/tracel-ai/burn, 2024. Version 0.14.0.

[26] Sebastian U. Stich. Local SGD Converges Fast and Communicates Little, May 2019. URL http://arxiv.org/abs/1805.09767. arXiv:1805.09767 [math].

[27] Sebastian U. Stich, Jean-Baptiste Cordonnier, and Martin Jaggi. Sparsified SGD with Memory, November 2018. URL http://arxiv.org/abs/1809.07599. arXiv:1809.07599 [cs].

[28] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding, November 2023. URL http://arxiv.org/abs/2104.09864. arXiv:2104.09864 [cs].

[29] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models, February 2023. URL http://arxiv.org/abs/2302.13971. arXiv:2302.13971 [cs].

[30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023. URL http://arxiv.org/abs/1706.03762. arXiv:1706.03762 [cs].

[31] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization, October 2019. URL http://arxiv.org/abs/1910.07467. arXiv:1910.07467 [cs].

[32] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel, September 2023. URL http://arxiv.org/abs/2304.11277. arXiv:2304.11277 [cs].